FUTURE GAMES WARSAW

Data Structures and Algorithms Final assignment

Marcel Triebe (GP26)

November 13th, 2026

Search Algorithm Selection, Suspect Identification, and Advanced Data Structure Implementation

For my Week 4 task, I chose Binary Search that was introduced early during the course. I implemented it from the first day we got to know it, and applied it to search through the available employee data.

The range of suspects I identified included everyone who entered or exited the building, with special attention to employees who both entered and left on the same day (August 9th, 2234). These employees were relevant to the investigation, especially one that entered and left within a minute.

For my Week 5 advanced data structure, I implemented a HashMap with collision handling using Lists as buckets. This makes sure that colliding elements are stored rather than overwritten. I reused the existing JSON reader and modified the task to store employee data in either my own HashMap implementation (which includes an integrated search function), or a standard array implementation, which makes it possible to apply different sorting and searching algorithms like Binary Search, Merge Sort, and Quick Sort.

Challenges and How I Overcame Them

In week 1, my main problem was to adapt a generic queue to the task system. I had to rethink how the generic system works, and then I connected it to task scripts so each dequeue executed the next task in order.

In week 2, I needed to load JSON without hardcoded paths. I didn't know how to make it not hardcoded, but I solved this by using a serialized TextAsset reference in combination with the Newtonsoft package.

For week 3 most difficult thing was the Quicksort, because I struggled to understand partitioning. Reviewing the lecture slides, pseudocode and additional resources helped me understand and implement it.

During week 4, I struggled to extend binary search to a date range. After some trial and error, I used two searches to find the lower and upper bounds in the sorted array, then returned the employees between them.

Finally, at week 5, retrieving date ranges from an unsorted hash map was challenging. I checked every bucket and its list for matching employees. This worked, but showed that the HashMap was less fitting for this data set.

Generic Search Implementation and Hash Function Selection (for VG)

I basically rewrote my first implementation of Binary Search, which originally used the standard DateTime type as input, and replaced the date and time values with generic types T and U. My goal here was to make the implementation more flexible so that it could work with different types of data instead of being restricted to DateTime values.

To make this work, I had to research how to make Binary Search understand the data set so that it could compare the values correctly. During this process, I found the IComparable interface, which allows different data types to define how their values should be compared. This helped me a lot, because it fixed the most important problem.
By specifying this requirement before using Binary Search, I could make sure that the algorithm received the necessary information to perform comparisons. This was an important part of making the implementation generic while keeping the original Binary Search functionality.

I implemented the HashMap in Week 5. I chose modulo and FNV-1a as my hashing functions. I continued using modulo because it was shown during the lectures and was a fast and simple way to distribute elements in the HashMap. It was also kind of straightforward to implement and understand. However, since modulo hashing alone is not very effective at preventing collisions, I also used the FNV-1a hashing function to reduce the possibility of

collisions as much as possible. I chose this approach because it provides a more structured way of generating hash values from input data.

Combining these approaches gave me a better and more flexible way of handling how the hash function is applied. It also allowed me to explore different hashing techniques and better understand their role in implementing a HashMap.